SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.
(BL3)

Assessment Tool Weightage: ***60%***

QUESTIONS: 6 **Total Marks:60**

Annexure A

Application Based Questions

Code of Conduct:

I V V Naveen Kumar/ ***192321018****certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
VV Naveenkumar

Annexure A

Application Based Questions

1. Use Case Diagram Analysis

A **Smart Hotel Reservation System** allows guests to search rooms, make reservations, cancel bookings, and process payments. Managers handle room allocation and reports, while payment services process transactions. The current design does not clearly distinguish actor responsibilities.

Analyze the system requirements, identify missing actors and use cases, and design an optimized Use Case Diagram showing proper relationships (include, extend, generalization) for better system clarity.

(10 Marks)

2. Class Diagram Design

A **University Management System** includes entities such as Student, Faculty, Course, Department, and Examination. The existing design lacks proper attributes, operations, and relationships among classes.

Analyze the system, identify relevant classes with attributes and methods, define relationships (association, aggregation, inheritance), and construct a well-structured Class Diagram.

(10 Marks)

3. Sequence Diagram Evaluation

In a **Vehicle Rental System**, a customer searches for a vehicle, books it, makes payment, and receives a rental confirmation. The interaction among system components is incomplete and poorly documented.

Analyze the process flow, identify missing objects and message sequences, and design a detailed Sequence Diagram representing proper interaction among system components.

(10 Marks)

4. State Diagram Construction

A **Traffic Signal Control System** operates through states such as Red Signal, Green Signal, Yellow Signal, Emergency Mode, and Maintenance Mode. The transitions between states are not clearly defined.

Analyze the system behavior, identify valid states and transitions, and construct a State Transition Diagram ensuring all events and conditions are properly represented.

(10 Marks)

5. Component Diagram Design

A **Smart Healthcare Portal** consists of modules such as Patient Management, Appointment Scheduling, Medical Records, Billing, and Notification Services. The current architecture does not clearly define component interactions and dependencies.

Analyze the system architecture, identify major components and their interactions, and design a Component Diagram showing dependencies and interfaces between modules.

(10 Marks)

6. Deployment Diagram Design

A **Smart City Waste Management System** is deployed across IoT waste-bin sensors, edge devices, cloud servers, and monitoring dashboards. The current design lacks clarity regarding physical architecture and communication links.

Analyze the deployment environment, identify nodes, artifacts, and communication links, and construct a detailed Deployment Diagram representing the complete system architecture.

(10 Marks)

RUBRICS FOR CALCULATION

Application Based Questions

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Annexure A

ANSWERS — APPLICATION BASED QUESTIONS

Name / Reg No: Naveen Kumar V V — 192321018

1. Use Case Diagram Analysis — Smart Hotel Reservation System

Actors identified:

• Guest (primary actor) – searches rooms, makes reservations, cancels bookings.

• Manager (primary actor) – allocates rooms, generates occupancy/revenue reports.

• Payment Service (secondary/supporting actor) – external system that processes transactions.

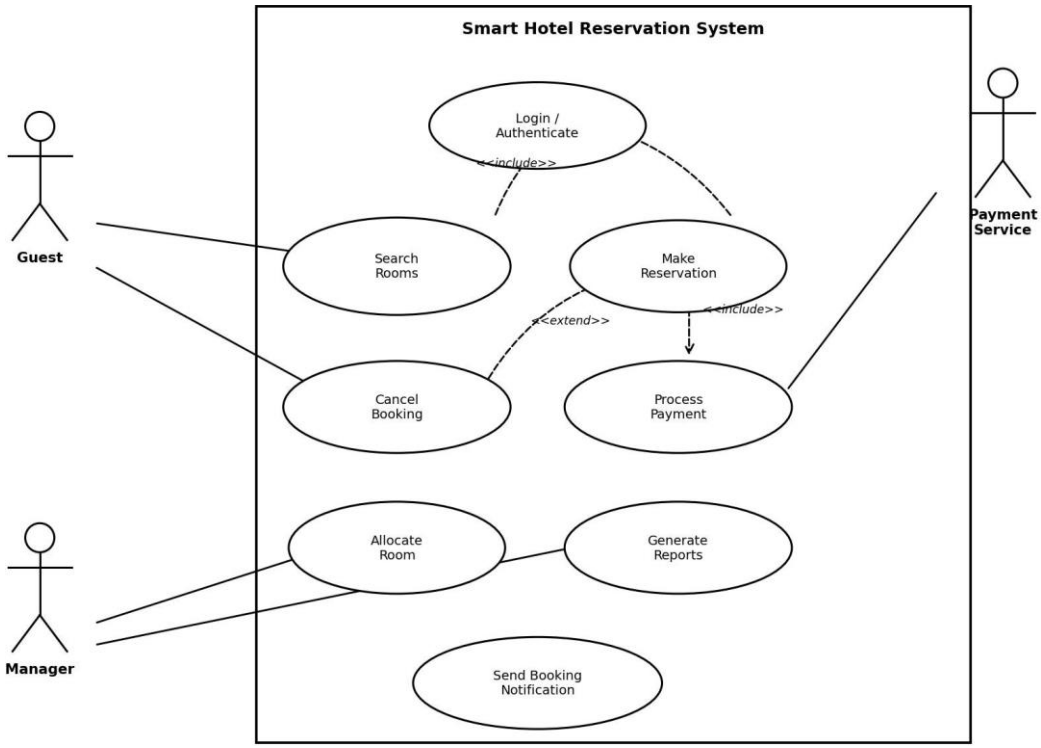
Missing use cases added: Login/Authenticate, Allocate Room, Send Booking Notification – these were implied but not explicit in the original design.

Relationships:

• <<include>>: “Search Rooms” and “Make Reservation” both include “Login/Authenticate”, since a guest must be signed in before any booking action.

• <<include>>: “Make Reservation” includes “Process Payment” – a reservation is only confirmed once payment is completed.

• <<extend>>: “Cancel Booking” extends “Make Reservation” – cancellation is an optional flow available only
af



gin/Authenticate behaviour.

Figure 1: Use Case Diagram — Smart Hotel Reservation System

This design clarifies actor boundaries — guests self-serve bookings, managers handle allocation/reporting, and payment logic is delegated to an external service — improving maintainability.

2. Class Diagram Design — University Management System

Classes with attributes and methods:

• Student: studentID, name, dept, semester | register(), viewResult()

• Faculty: facultyID, name, designation | assignCourse(), enterMarks()

• Course: courseID, title, credits | addCourse(), updateSyllabus()

• Department: deptID, deptName | addFaculty()

• Examination: examID, examDate, marks | scheduleExam(), publishResult()

• UGStudent / PGStudent (specializations of Student): add programme-specific attributes and viewCurriculum().

Relationships:

• Association: Faculty teaches Course; Student appears in Examination; Course is evaluated in Examination.

• Aggregation: Department aggregates Faculty and Course – both can exist as records even if reassigned, but are organized under a Department.

• Inheritance/Generalization: UGStudent and PGStudent inherit from Student, each specializing curriculum details.

Multiplicities: Faculty (1) – Course (0..*); Student (1) – Examination (0..*); Course (1) – Examination (0..*).

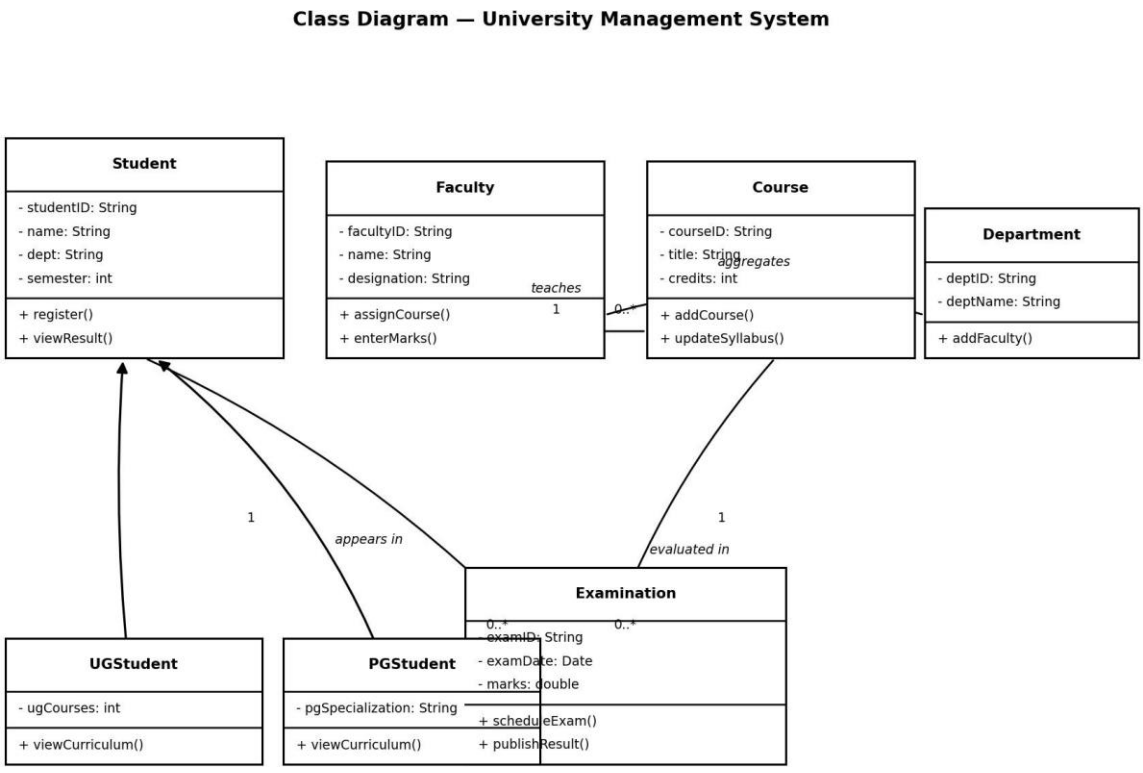


Figure 2: Class Diagram — University Management System

Modelling Examination as its own class (rather than an attribute of Student or Course) keeps result history independently traceable across multiple courses and semesters.

3. Sequence Diagram Evaluation — Vehicle Rental System

Objects involved: Customer, :RentalUI (boundary), :RentalController (control), :PaymentGateway (external system), :VehicleInventory (entity).

Message flow:

• 1. searchVehicle() 2. checkAvailability() 3. getVehicleStatus() 4. availabilityResult() 5. displayResults() 6. bookVehicle() 7. requestPayment() 8. processPayment() 9. paymentConfirmed() 10. reserveVehicle() 11. rentalConfirmation() 12. displayConfirmation().

Activation and lifelines: each object has a dashed vertical lifeline; activation bars are shown on :RentalController (active throughout availability-checking, booking and payment coordination), on :PaymentGateway (active only while processing payment), and on :VehicleInventory (active while confirming/reserving the vehicle).

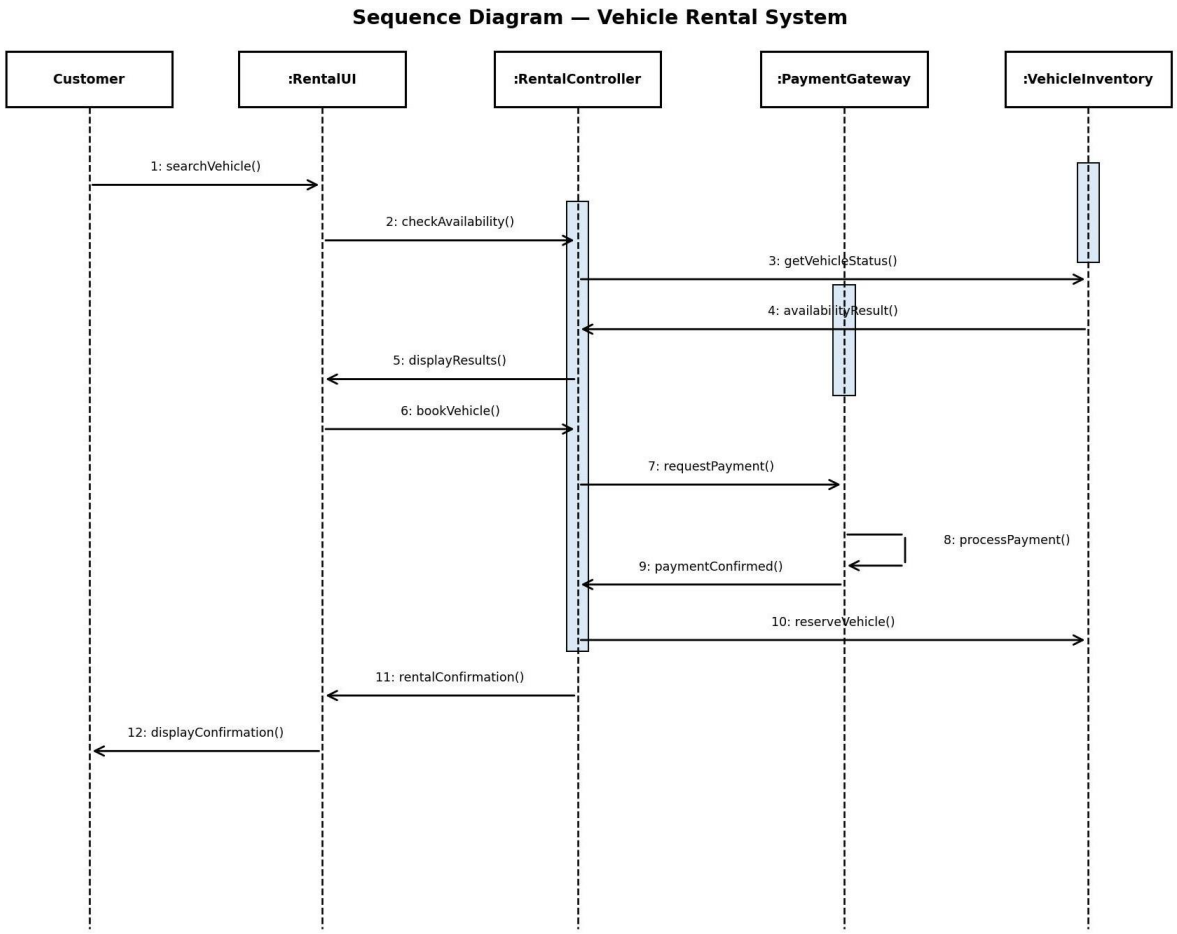


Figure 3: Sequence Diagram — Vehicle Rental System

Adding the missing availability-check round-trip (steps 2–5) before booking prevents double-booking of the same vehicle, which was not evident in the original incomplete flow.

4. State Diagram Construction — Traffic Signal Control System

States identified: Red Signal, Green Signal, Yellow Signal, Emergency Mode, Maintenance Mode.

Events and transitions:

• Red → Green → Yellow → Red: each triggered by timerExpires(), forming the normal operating cycle.

• Any normal state → Emergency Mode: triggered by emergencyVehicleDetected(); Emergency Mode → normal cycle: triggered by emergencyCleared().

• Any state → Maintenance Mode: triggered by maintenanceTriggered(), used by technicians to safely halt automatic switching.

Initial/final states: a filled black circle marks system start (defaulting into Red Signal); a circled dot marks the final state, reached when Maintenance Mode completes and the controller is powered down or fully reset.

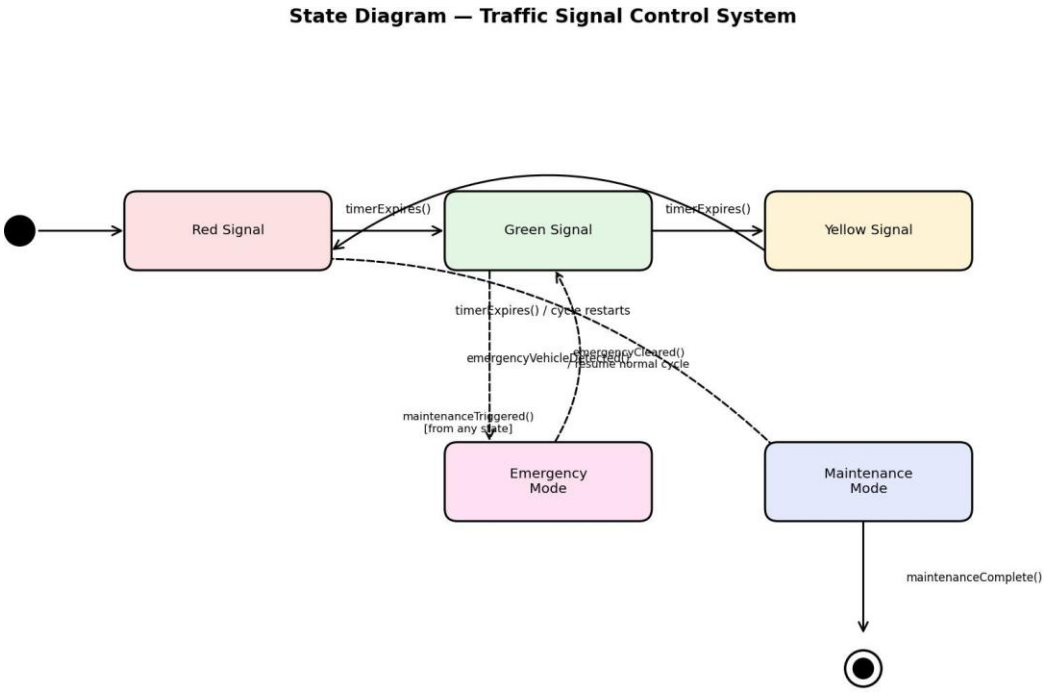


Figure 4: State Diagram — Traffic Signal Control System

Giving Emergency Mode and Maintenance Mode transitions from every normal state (rather than only from Red) ensures the controller can react immediately regardless of which phase of the cycle it is in.

5. Component Diagram Design — Smart Healthcare Portal

Major components: Patient Management, Appointment Scheduling, Medical Records, Billing, Notification Service, API Gateway, Database Service.

Interfaces and interactions: Patient Management provides a patient-lookup interface consumed by Appointment Scheduling and Medical Records; API Gateway exposes a unified REST interface used by all front-facing modules; Database Service provides a persistence interface used by every module that stores data.

Dependencies: Appointment Scheduling depends on Patient Management (to verify patient identity) and Notification Service (to send reminders); Medical Records depends on Patient Management and Database Service; Billing depends on Medical Records (for

billable services) and Database Service; all modules route external calls through the API Gateway.

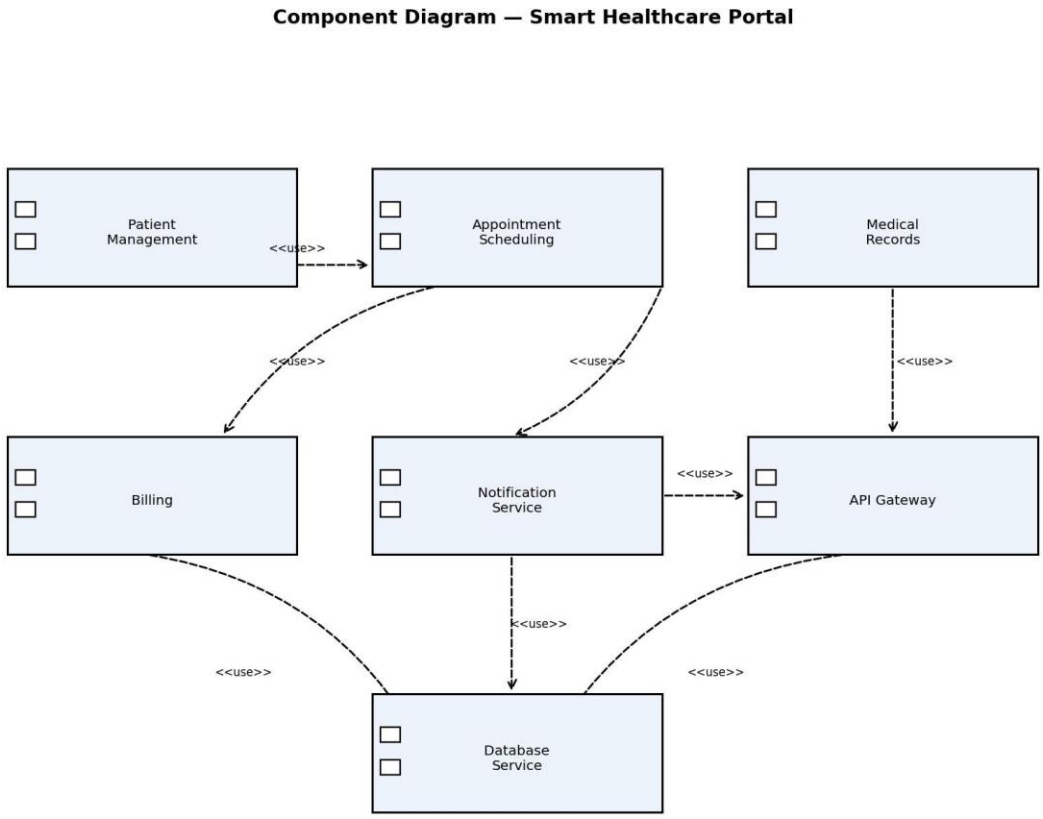


Figure 5: Component Diagram — Smart Healthcare Portal

Routing Billing through Medical Records (rather than letting it read raw patient data directly) keeps clinical and financial concerns cleanly separated, satisfying typical healthcare data-access policies.

6. Deployment Diagram Design — Smart City Waste Management System

Nodes identified: IoT Waste-Bin Sensor Node, Edge Device, Cloud Server, Monitoring Dashboard (city authority's workstation).

Artifacts per node:

• IoT Waste-Bin Sensor Node — Fill-Level Firmware (measures bin fill percentage).

• Edge Device — Data Aggregator (buffers and filters readings from nearby bins).

• Cloud Server — Route Optimizer, Waste Database, Alert Service.

• Monitoring Dashboard — browser-based Web Dashboard Application.

Communication links: Sensor Node ↔ Edge Device over LoRaWAN/NB-IoT (low-power wide-area network); Edge Device ↔ Cloud Server over HTTPS/TCP-IP; Cloud

Server ↔ Monitoring Dashboard over HTTPS (REST API) for live fill-level maps, optimized collection routes and overflow alerts.

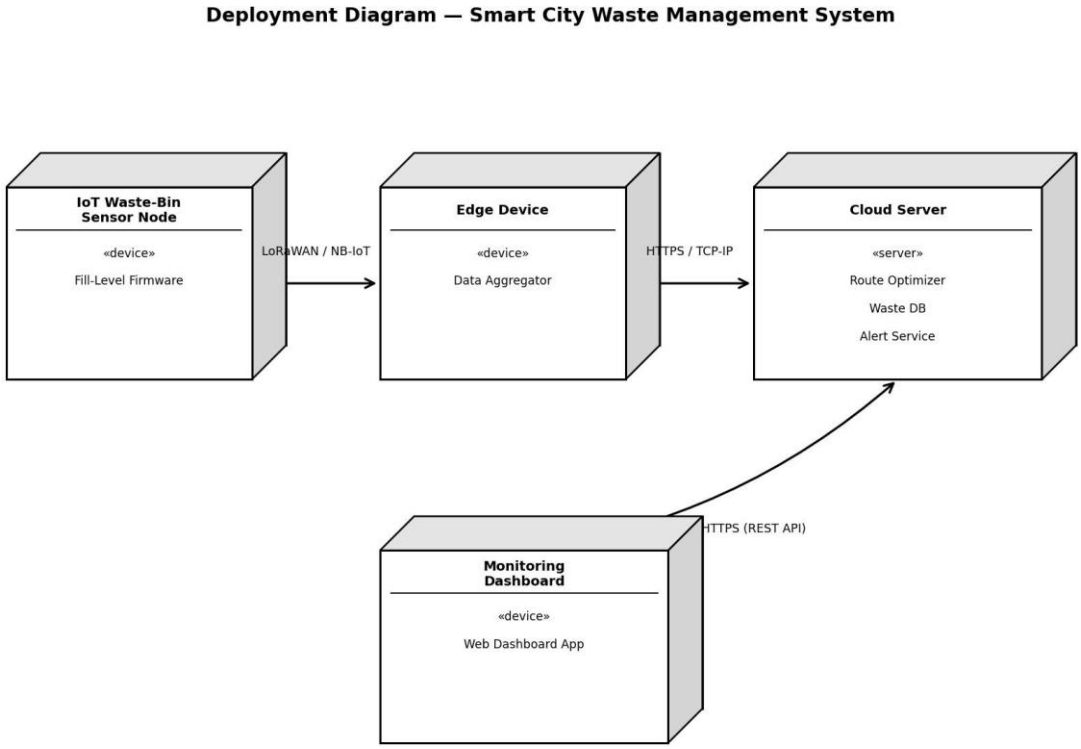


Figure 6: Deployment Diagram — Smart City Waste Management System

An Edge Device aggregating multiple sensor nodes before sending data to the Cloud Server reduces the number of long-range transmissions needed, which is important given .